

React JS

Complete Beginner's Guide

A JavaScript library by Facebook (Meta) for building user interfaces

01 What is React JS?

React JS is a free, open-source JavaScript library made by Facebook (Meta). It is used to build the visual part (User Interface) of websites and web apps — things the user sees and clicks on. React makes it easy to build fast, interactive UIs by breaking the page into small reusable pieces called **components**.

02 Traditional UI vs React JS UI

Feature	Traditional UI	React JS UI
Language	HTML + CSS + JS (separate)	JSX — HTML & JS together
Update UI	You do it manually	React does it automatically
Code style	Gets messy over time	Clean and organised
Reuse code	Not easy	Very easy with components
Page reload	Often reloads	No reloads — stays on one page
Maintenance	Hard to update	Easy to update and grow

03 Why Use React JS?

Reusable Components	Fast Performance
Build a button or card once — use it everywhere. Saves time and keeps code tidy.	React only updates the part of the page that changed. The rest stays untouched.
Automatic UI Updates	No Full Page Reloads
When your data changes, React automatically refreshes the screen. No manual work needed.	The page never fully reloads. Users get a smooth, app-like experience.
Organised Code	React Native

Code is split into small components. Easy to find, fix, and improve.

Use the same React skills to build mobile apps for iOS and Android.

04 Imperative vs Declarative

What is the difference? Imperative means telling the computer *how* to do every step. Declarative means telling the computer *what* you want — it figures out the steps.

Imperative Step-by-step directions	Declarative Just say the destination
"Go straight, turn left at the signal, take the second right, park at the third building..."	"Take me to Domino's Pizza."

	Imperative	Declarative
Approach	Tell how to do every step	Just describe the result you want
DOM Updates	You write the update code yourself	React writes the updates for you
Code	You write more lines	You write fewer lines
Easy to read?	Harder to read	Easier to read
Example language	Vanilla JavaScript	React, HTML, SQL

Demo — Vanilla JS:
codesandbox.io/p/sandbox/vanilla-js-demo-6049kj

Demo — React JS:
codesandbox.io/p/sandbox/react-vs-vanilla-demo-uc08fv

05 Vite — Frontend Build Tool

What is Vite? Vite is a tool that helps you set up and run React projects. It is much faster than older tools like Webpack or Create React App (CRA). It does two main jobs:

Instant Server Start	Development	Build
Starts instantly when you run the project — no waiting, even for large apps.	Shows your code changes live in the browser when you save the file in your editor. No need to refresh.	Creates an optimised version of your project ready to upload to the internet.

How to create a Vite project:

```
npm create vite@latest
```

Key Features of Vite:

1. Works Out of the Box

These features are already included in Vite. You do NOT need to install them separately:

TypeScript Support	JSX Support
Write TypeScript code directly — no extra setup needed.	Use JSX (HTML inside JavaScript) right away in .jsx files.
CSS Modules	Image Import
Scope CSS to a single component so styles never clash globally.	Import images directly into your JS/JSX files like any other module.
Environment Variables	
.env files work out of the box to store secret keys and settings.	

2. Optimised Build for a Fast Website

When you are ready to publish your project, Vite automatically does all of this:

Removes Unused Code	Reduces Code Size
Dead code (scripts that are never used) is deleted from the final bundle.	Code is minified — whitespace and long names are shortened to save bytes.
Code Splitting	Asset Optimisation
The app is split into smaller chunks so each page only loads what it needs.	Images and files are compressed and optimised for faster downloads.

3. Full CSS Features — Works Automatically

All modern CSS tools are supported by Vite without any manual configuration:

CSS Modules	Sass / LESS
Locks CSS inside a component — styles will NOT leak into other parts of the page.	Write advanced CSS with variables, nesting, and mixins using Sass or LESS.
PostCSS	CSS Code Splitting
Automatically optimises your CSS — adds vendor prefixes, removes unused rules.	Only the CSS needed for the current page is loaded, keeping things fast.

4. Server-Side Rendering (SSR)

With SSR, the page is built on the server before it reaches the browser — great for speed and SEO:

What is SSR?

The server generates the full HTML page and sends it to the browser, not just JavaScript.

Faster First Load

Users see content immediately without waiting for JavaScript to run on their device.

Better SEO

Search engines can read the full page content since it is pre-rendered as HTML.

Build Your Own

Vite lets you set up your own SSR pipeline or integrate with frameworks like Next.js.

5. NPM Dependency Pre-Bundling

Vite prepares (pre-bundles) your npm libraries in advance using a tool called esbuild:

Ready Immediately

Libraries are bundled once at startup and cached — they load instantly every time after.

Faster Page Load

Pre-bundled packages are served as a single efficient file, reducing network requests.

Faster Development

Because libraries are ready ahead of time, your coding workflow is smoother and quicker.

Automatic

Vite detects your dependencies and pre-bundles them automatically — nothing to configure.

6. Plugin System — Add New Features via vite.config.js

Vite can be extended with plugins. You add them to vite.config.js to get extra features in your project:

@vitejs/plugin-react

Adds full React support — JSX transform, Fast Refresh, and React-specific optimisations.

vite-plugin-pwa

Turns your web app into a Progressive Web App (PWA) — works offline, installable on phone.

vite-plugin-checker

Shows TypeScript errors directly in the browser while you code, so you catch mistakes instantly.